

TALENTX

FULL PROJECT DOCUMENTATION

1. Project Overview

TALENTX is a talent-first recruitment platform intended to move the process from reactive applications to proactive discovery. Candidates create one verified Talent Passport; employers search and contact suitable candidates; candidates retain visibility and response control.

- Candidate-first, employer-discovery model
- Evidence-backed talent profiles
- Explainable skill-level matching
- Consent-driven visibility
- Full-stack academic MVP with clear extension points
- Post-match project governance through contracts, milestones, escrow, deliverables and project health.
- Employer Challenges that create a practical cold-start and revenue mechanism.
- Clear extension points for future verification, AI assistance and institutional integrations.

2. Problem Statement

- Application black holes: candidates can submit many applications without useful feedback.
- Screening overload: employers must sort large applicant pools.
- Keyword filtering can miss capable candidates.
- Self-reported skills lack strong evidence.
- Candidates have limited control over how and when employers discover them.

The supplied deck frames the problem around application volume, screening overload and system misalignment. [filecite?turn0file0?L37-L53?](#)

3. Product Positioning

Area	Traditional application / reverse recruiting	TALENTX
Candidate cost	Applications free; agencies may charge	Free for candidates
Core action	Candidate applies / agency applies	Employer discovers candidate
Profile	Resume/self-report	Verified Talent Passport
Proof	Limited	GitHub, certifications, assessments, projects, internships
Match explanation	Usually absent	Per-skill breakdown
Candidate control	Low	Visibility + accept/decline/negotiate

The source explicitly distinguishes TALENTX from paid reverse-recruiting agencies and defines its model as talent-first discovery. [filecite?url=turn0file0L61-L73](#)

Competitive framing: TALENTX sits at the intersection of LinkedIn Recruiter-style discovery, Upwork-style project execution, and Toptal-style quality signals, while integrating evidence, explainability and governed delivery in one workflow.

The differentiation claim should remain precise: TALENTX is not presented as the literal replacement for every capability of these platforms; its advantage is the integrated proof → discovery → delivery loop.

4. Major Features

4.1 Candidate Features

- Registration and secure login
- Talent Passport creation/editing
- Skills, projects, certifications, assessments and internship records
- Evidence links / verification badges
- Availability and visibility controls
- Opportunity invitations
- Accept, decline or negotiate
- Explainable match report and skill gaps
- Secure in-platform messaging

4.2 Employer Features

- Employer registration and login
- Opportunity/job creation
- Required and preferred skills
- Candidate discovery search
- Filters and re-ranking
- Explainable match report
- Targeted invitations
- Messaging and response tracking

4.3 Admin Features

- Role-protected admin access
- User and account oversight
- Verification/moderation review
- Opportunity moderation
- Reports/flags
- Audit logs
- System metrics
- Bias-test review and account controls
- Post-match project oversight: milestones, deliverables, escrow status and project health.
- Dispute resolution using project evidence, chat history and deliverable versions.

5. Core Product Modules

Module	Purpose	Primary user
Authentication	Registration, login, JWT, roles	All
Talent Passport	Verified profile and evidence	Candidate
Discovery	Search/filter candidate pool	Employer
Opportunity	Create/manage hiring opportunities	Employer
Matching	Score and rank candidates	Employer/Candidate
Messaging	Invitation and communication flow	Candidate/Employer
Admin	Moderation, audit and oversight	Admin
Verification	Evidence and source status	Candidate/Admin
Project Workspace	Convert an accepted match into a governed project contract	Candidate/Employer
Milestone Tracker	Phased deliverables, deadlines and monetary weights	Candidate/Employer
Escrow Vault	Hold and release milestone-linked project funds	Candidate/Employer
Deliverable Manager	Versioned v1/v2/v3 submissions and revisions	Candidate/Employer
Health Engine	Green / Yellow / Red project health from deadlines and budget usage	Candidate/Employer
Employer Challenges	Paid micro-tasks for cold-start activation and revenue	Employer/Candidate
Dispute Resolution	Evidence-based admin mediation	Admin

6. System Architecture

Recommended architecture: React frontend → REST API → Spring Boot controllers → service layer → MongoDB collections. The supplied deck identifies React.js, Spring Boot, MongoDB, Spring Security/JWT, GitHub API integration, Maven, Postman, RESTful APIs and Bean Validation as key tooling. [filecite?turnOfffile0?L209-L230?](#)

Layer	Technology	Responsibility
Frontend	React.js + HTML5/CSS3	Pages, components, forms, dashboards, API calls
API	Spring Boot	REST endpoints, validation, authentication entry points
Business	Service layer	Matching, passport, opportunity, invitation rules
Persistence	MongoDB	Users, passports, opportunities, matches, messages
Security	Spring Security + JWT	Authentication, roles, protected endpoints
Testing	Postman + automated tests	API and integration verification

7. Complete Project Folder Blueprint

The following master repository blueprint is the implementation reference for TALENTX. It covers the complete frontend, backend, admin portal, database, API testing, documentation, screenshots, and deployment structure.

TALENTX/

- ├── README.md
- ├── .gitignore
- ├── .env.example
- ├── docker-compose.yml
- ├── docs/
 - ├── TALENTX_Project_Blueprint.pdf
 - ├── TALENTX_Project_Documentation.pdf
 - ├── API_DOCUMENTATION.md
 - ├── DATABASE_DOCUMENTATION.md
 - ├── ARCHITECTURE.md
 - ├── SECURITY.md
 - └── DEPLOYMENT.md
- ├── frontend/
 - ├── package.json
 - ├── vite.config.js
 - ├── index.html
 - ├── public/
 - ├── favicon.ico
 - ├── logo.svg
 - └── images/
 - └── src/
 - ├── main.jsx
 - ├── App.jsx
 - ├── assets/
 - ├── images/
 - ├── icons/
 - └── fonts/
 - ├── components/
 - ├── common/
 - ├── Navbar.jsx
 - ├── Footer.jsx
 - ├── Button.jsx
 - ├── Input.jsx
 - ├── Modal.jsx
 - ├── Loader.jsx
 - ├── Badge.jsx
 - ├── EmptyState.jsx
 - └── Pagination.jsx
 - ├── candidate/
 - ├── TalentPassportCard.jsx
 - ├── SkillCard.jsx
 - ├── ProjectCard.jsx
 - ├── CertificationCard.jsx
 - ├── AssessmentCard.jsx
 - ├── EvidenceCard.jsx
 - ├── VerificationBadge.jsx
 - ├── OpportunityCard.jsx
 - └── ProfileCompletion.jsx
 - ├── employer/
 - ├── CompanyCard.jsx
 - ├── OpportunityCard.jsx
 - ├── CandidateCard.jsx
 - ├── CandidatePreview.jsx
 - ├── SearchFilters.jsx
 - ├── InvitationCard.jsx
 - └── TalentTable.jsx
 - └── matching/
 - ├── MatchScore.jsx
 - ├── MatchBreakdown.jsx
 - └── SkillMatch.jsx

- └─ EvidenceBreakdown.jsx
- └─ MatchReason.jsx
- └─ SkillGap.jsx

- └─ messaging/

- └─ ConversationList.jsx
- └─ ChatWindow.jsx
- └─ MessageBubble.jsx
- └─ MessageInput.jsx

- └─ layouts/

- └─ PublicLayout.jsx
- └─ AuthLayout.jsx
- └─ CandidateLayout.jsx
- └─ EmployerLayout.jsx

- └─ pages/

- └─ public/

- └─ Home.jsx
- └─ About.jsx
- └─ HowItWorks.jsx
- └─ ForCandidates.jsx
- └─ ForEmployers.jsx
- └─ Contact.jsx

- └─ auth/

- └─ Login.jsx
- └─ CandidateRegister.jsx
- └─ EmployerRegister.jsx
- └─ ForgotPassword.jsx
- └─ VerifyAccount.jsx

- └─ candidate/

- └─ Dashboard.jsx
- └─ TalentPassport.jsx
- └─ EditPassport.jsx
- └─ Skills.jsx
- └─ Projects.jsx
- └─ GithubProof.jsx
- └─ Certifications.jsx
- └─ Assessments.jsx
- └─ AssessmentAttempt.jsx
- └─ AssessmentResult.jsx
- └─ VerificationCenter.jsx
- └─ Opportunities.jsx
- └─ OpportunityDetails.jsx
- └─ Messages.jsx
- └─ Notifications.jsx
- └─ PrivacySettings.jsx

- └─ employer/

- └─ Dashboard.jsx
- └─ CompanyProfile.jsx
- └─ CreateOpportunity.jsx
- └─ ManageOpportunities.jsx
- └─ TalentDiscovery.jsx
- └─ CandidateProfile.jsx
- └─ MatchReport.jsx
- └─ Invitations.jsx
- └─ Messages.jsx

- └─ services/

- └─ api.js
- └─ authService.js
- └─ passportService.js
- └─ skillService.js
- └─ projectService.js
- └─ evidenceService.js
- └─ certificationService.js
- └─ assessmentService.js
- └─ opportunityService.js
- └─ discoveryService.js
- └─ matchService.js

- ├─ invitationService.js
- ├─ messageService.js
- └─ notificationService.js
- ├─ context/
 - ├─ AuthContext.jsx
 - ├─ UserContext.jsx
 - └─ NotificationContext.jsx
- ├─ hooks/
 - ├─ useAuth.js
 - ├─ useApi.js
 - └─ useNotification.js
- ├─ routes/
 - ├─ AppRoutes.jsx
 - ├─ ProtectedRoute.jsx
 - ├─ CandidateRoute.jsx
 - └─ EmployerRoute.jsx
- ├─ utils/
 - ├─ validators.js
 - ├─ formatters.js
 - ├─ constants.js
 - └─ storage.js
- └─ styles/
 - ├─ index.css
 - ├─ variables.css
 - └─ responsive.css

backend/

- └─ talentx-api/
 - ├─ pom.xml
 - └─ src/
 - ├─ main/
 - ├─ java/
 - └─ com/
 - └─ talentx/
 - ├─ TalentXApplication.java
 - ├─ controller/
 - ├─ AuthController.java
 - ├─ PassportController.java
 - ├─ SkillController.java
 - ├─ ProjectController.java
 - ├─ EvidenceController.java
 - ├─ CertificationController.java
 - ├─ AssessmentController.java
 - ├─ OpportunityController.java
 - ├─ DiscoveryController.java
 - ├─ MatchController.java
 - ├─ InvitationController.java
 - ├─ MessageController.java
 - └─ NotificationController.java
 - ├─ service/
 - ├─ AuthService.java
 - ├─ PassportService.java
 - ├─ SkillService.java
 - ├─ ProjectService.java
 - ├─ EvidenceService.java
 - ├─ CertificationService.java
 - ├─ AssessmentService.java
 - ├─ OpportunityService.java
 - ├─ DiscoveryService.java
 - └─ MatchService.java

```
├─ InvitationService.java
├─ MessageService.java
├─ NotificationService.java

repository/
├─ UserRepository.java
├─ TalentPassportRepository.java
├─ SkillRepository.java
├─ ProjectRepository.java
├─ EvidenceRepository.java
├─ CertificationRepository.java
├─ AssessmentRepository.java
├─ AssessmentAttemptRepository.java
├─ OpportunityRepository.java
├─ MatchRepository.java
├─ InvitationRepository.java
├─ ConversationRepository.java
├─ MessageRepository.java
├─ NotificationRepository.java

model/
├─ User.java
├─ TalentPassport.java
├─ Skill.java
├─ Project.java
├─ Evidence.java
├─ Certification.java
├─ Assessment.java
├─ AssessmentAttempt.java
├─ Opportunity.java
├─ Match.java
├─ Invitation.java
├─ Conversation.java
├─ Message.java
├─ Notification.java

dto/
├─ auth/
│   ├── RegisterRequest.java
│   ├── LoginRequest.java
│   └─ AuthResponse.java
├─ candidate/
│   ├── PassportRequest.java
│   ├── SkillRequest.java
│   ├── ProjectRequest.java
│   └─ EvidenceRequest.java
├─ employer/
│   ├── OpportunityRequest.java
│   └─ CompanyRequest.java

mapper/
├─ UserMapper.java
├─ PassportMapper.java
├─ OpportunityMapper.java

security/
├─ SecurityConfig.java
├─ JwtService.java
├─ JwtAuthenticationFilter.java
├─ CustomUserDetailsService.java
├─ PasswordConfig.java

matching/
├─ MatchEngine.java
├─ ScoreCalculator.java
├─ SkillMatcher.java
├─ ProjectRelevanceEvaluator.java
├─ AssessmentEvaluator.java
├─ EvidenceEvaluator.java
├─ ExplanationBuilder.java
```

```
├── verification/
│   ├── VerificationService.java
│   ├── EvidenceVerifier.java
│   └── VerificationStatus.java
├── integration/
│   └── github/
│       ├── GithubService.java
│       └── GithubRepositoryData.java
├── validation/
│   ├── PasswordValidator.java
│   ├── OpportunityValidator.java
│   └── EvidenceValidator.java
├── exception/
│   ├── GlobalExceptionHandler.java
│   ├── ResourceNotFoundException.java
│   ├── UnauthorizedException.java
│   └── ValidationException.java
├── config/
│   ├── CorsConfig.java
│   ├── MongoConfig.java
│   └── AppConfig.java
├── resources/
│   ├── application.properties
│   └── application-dev.properties
├── admin-portal/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   ├── com/
│       │   │   │   ├── talentx/
│       │   │   │   │   ├── admin/
│       │   │   │   │   │   ├── controller/
│       │   │   │   │   │   │   ├── AdminLoginServlet.java
│       │   │   │   │   │   │   ├── DashboardServlet.java
│       │   │   │   │   │   │   ├── UserManagementServlet.java
│       │   │   │   │   │   │   ├── CandidateVerificationServlet.java
│       │   │   │   │   │   │   ├── EmployerVerificationServlet.java
│       │   │   │   │   │   │   ├── OpportunityManagementServlet.java
│       │   │   │   │   │   │   ├── ReportsServlet.java
│       │   │   │   │   │   │   └── AnalyticsServlet.java
│       │   │   │   │   │   ├── service/
│       │   │   │   │   │   │   ├── AdminService.java
│       │   │   │   │   │   │   ├── VerificationService.java
│       │   │   │   │   │   │   ├── ModerationService.java
│       │   │   │   │   │   │   └── AnalyticsService.java
│       │   │   │   │   │   └── model/
│       │   │   │   │   │   │   ├── AdminUser.java
│       │   │   │   │   │   │   ├── VerificationRecord.java
│       │   │   │   │   │   │   └── AuditLog.java
│       │   │   │   │   └── webapp/
│       │   │   │   │   │   ├── WEB-INF/
│       │   │   │   │   │   │   └── web.xml
│       │   │   │   │   │   ├── css/
│       │   │   │   │   │   │   ├── admin.css
│       │   │   │   │   │   │   ├── dashboard.css
│       │   │   │   │   │   │   └── tables.css
```



```

├── js/
│   └── admin.js
├── views/
│   ├── login.jsp
│   ├── dashboard.jsp
│   ├── users.jsp
│   ├── candidate-verification.jsp
│   ├── employer-verification.jsp
│   ├── opportunities.jsp
│   ├── reports.jsp
│   └── analytics.jsp
├── database/
│   ├── collections/
│   │   ├── users.json
│   │   ├── skills.json
│   │   ├── assessments.json
│   │   └── opportunities.json
│   ├── seed/
│   │   ├── users.js
│   │   ├── candidates.js
│   │   ├── employers.js
│   │   ├── opportunities.js
│   │   └── assessments.js
│   └── indexes/
│       └── create-indexes.js
├── postman/
│   ├── TALENTX_API.postman_collection.json
│   └── TALENTX_ENVIRONMENT.postman_environment.json
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── api/
│   └── test-data/
├── screenshots/
│   ├── candidate/
│   ├── employer/
│   └── admin/
├── deployment/
├── Dockerfile.frontend
├── Dockerfile.backend
├── Dockerfile.admin
├── nginx.conf
└── deployment-notes.md

```

Blueprint source: user-provided TALENTX folder blueprint. Use this structure as the master file/folder reference during implementation.

8. Frontend File Blueprint

Path/File	Responsibility
App.jsx	Root routes and app shell
pages/Login.jsx	Candidate/employer/admin login
pages/Register.jsx	Registration
pages/CandidateDashboard.jsx	Candidate overview

pages/Passport.jsx	Passport builder and evidence
pages/EmployerDashboard.jsx	Employer overview
pages/Discovery.jsx	Candidate search and filters
pages/MatchDetails.jsx	Explainable score
pages/Messages.jsx	Messaging
components/SkillBadge.jsx	Skill/evidence status
components/MatchScore.jsx	Score visualization
components/OpportunityCard.jsx	Opportunity summary
services/api.js	HTTP client/base URL
services/authService.js	Token/session helpers
context/AuthContext.jsx	Authentication state

9. Backend File Blueprint

Package	Example files	Rule
controller	AuthController, PassportController, OpportunityController, MatchingController, MessageController, AdminController	Keep controllers thin
service	AuthService, PassportService, OpportunityService, MatchingService, NotificationService, AdminService	Business logic belongs here
repository	UserRepository, PassportRepository, OpportunityRepository, MatchRepository, MessageRepository	Mongo persistence
model	User, TalentPassport, Opportunity, Match, Message, Verification	Database/domain structures
dto	LoginRequest, RegisterRequest, PassportResponse, MatchResponse, OpportunityRequest	Do not expose entities directly
security	JwtFilter, JwtService, CustomUserDetailsService	Authentication and authorization
config	SecurityConfig, CorsConfig, MongoConfig	Environment/configuration
exception	GlobalExceptionHandler, ResourceNotFoundException, ValidationException	Centralized error handling

10. MongoDB Data Model

Collection	Key fields	Purpose
users	_id, name, email, passwordHash, role, status, createdAt	Identity/access
talent_passports	userId, headline, skills[],	Candidate evidence

	projects[], certifications[], assessments[], internships[], availability, visibility	
opportunities	employerId, title, description, requiredSkills[], preferredSkills[], status, createdAt	Hiring opportunities
matches	opportunityId, candidateId, totalScore, skillBreakdown[], evidence	Explainable match
messages	conversationId, senderId, receiverId, opportunityId, body, timestamp	Communication
audit_logs	actorId, action, targetId, timestamp, metadata	Security/admin audit
projects	matchId, employerId, freelancerId, title, totalBudget, status, healthScore, createdAt	Post-match project workspace
milestones	projectId, title, description, deadline, monetaryWeight, isCompleted, order	Phased deliverables
escrow_transactions	projectId, milestoneId, amount, payerId, payeeId, status, releaseDate	Milestone-linked fund state
deliverables	milestoneId, uploaderId, fileUrl, fileName, version, revisionNotes, status	Versioned deliverables
challenges	employerId, title, description, requiredSkills[], prizeAmount, deadline, status	Paid employer challenges
challenge_submissions	challengeId, candidateId, solutionUrl, submittedAt, isWinner, comments	Challenge solutions
disputes	projectId, raisedBy, reason, evidenceUrls[], adminNotes, resolution, resolvedAt	Conflict resolution

Use indexes for frequently queried fields such as email, role, opportunity status and searchable skill fields. Exact schema choices can evolve during implementation.

11. Matching Algorithm

The supplied concept specifies this illustrative weighting:

Component	Weight
Verified Skills	40%
Project Relevance	25%
Assessment Scores	20%
Certifications & Internships	10%
Profile Completeness	5%

A practical MVP implementation should normalize each component to 0–100 and compute a weighted total. The output should include the component scores and per-skill evidence, not only the final number.

Example: $\text{totalScore} = 0.40 \cdot \text{verifiedSkills} + 0.25 \cdot \text{projectRelevance} + 0.20 \cdot \text{assessmentScore} + 0.10 \cdot \text{certificationInternship} + 0.05 \cdot \text{profileCompleteness}$.

The supplied deck emphasizes that matching must be auditable and explainable to both employers and candidates. [filecite?turn0file0?L153-L157?](#)

12. API Blueprint

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Authenticate and issue JWT
GET	/api/passports/me	Get current candidate passport
PUT	/api/passports/me	Update passport
POST	/api/opportunities	Create opportunity
GET	/api/opportunities	List employer opportunities
GET	/api/discovery/candidates	Search/filter candidates
GET	/api/matches/{id}	Get explainable match
POST	/api/opportunities/{id}/invite	Send candidate invitation
POST	/api/messages	Send message
GET	/api/messages/{conversationId}	Read conversation
GET	/api/admin/users	Admin user list
POST	/api/projects	Create project contract after accepted match
GET	/api/projects/{id}	Get project workspace
POST	/api/projects/{id}/milestones	Add milestone
PUT	/api/milestones/{id}/complete	Complete milestone
POST	/api/projects/{id}/escrow/lock	Lock project funds
PUT	/api/escrow/{id}/release	Release milestone payment
POST	/api/milestones/{id}/deliverables	Upload versioned deliverable
GET	/api/deliverables/{id}/versions	Get deliverable versions
POST	/api/challenges	Create employer challenge
GET	/api/challenges	List open challenges
POST	/api/challenges/{id}/submit	Submit challenge solution
POST	/api/disputes	Raise dispute
PUT	/api/disputes/{id}/resolve	Admin resolves dispute

13. Authentication & Security

- JWT-based stateless authentication
- Role-based access: Candidate, Employer, Admin
- BCrypt password hashing
- Bean Validation for request payloads
- CORS restricted to approved frontend origins
- HTTPS in deployment
- No secrets in Git
- Environment variables for DB credentials, JWT secret and external API tokens
- Centralized exception handling
- Audit sensitive admin actions

These principles align with the source deck's security and privacy section, including JWT, BCrypt, validation and consent-driven visibility. [filecite2turn0file0L238-L248](#)

14. Privacy & Consent

- Talent Passports should be private by default.
- Candidate chooses whether to become discoverable.
- Employer receives only information permitted by visibility rules.
- Candidate can accept, decline or negotiate an invitation.
- Provide account/data deletion controls in the MVP where feasible.
- Avoid storing unnecessary personal data.
- Do not expose private profile fields through public endpoints.

The source concept explicitly calls for opt-in visibility and platform-level consent before sharing.

[filecite2turn0file0L247-L269](#)

15. Responsible AI / Matching Governance

- Keep the MVP rule-based and explainable.
- Avoid structural penalties for candidates without paid credentials.
- Cap certification/internship contribution as defined in the concept.
- Expose per-skill evidence and score components.
- Run pre-pilot fairness/bias checks on synthetic or test data.
- Document model limitations and avoid claiming that the score predicts hiring success.

The supplied deck labels this section 'Responsible AI' and proposes bias auditing before launch.

[filecite2turn0file0L256-L269](#)

16. What to Use vs What Not to Use

Use	Avoid / Restrict
React for main candidate/employer UI	JSP pages for the main UI when React is selected
Spring Boot REST APIs	Servlet/JSP controllers for core business logic
MongoRepository / MongoTemplate	JDBC/SQL DAOs for MongoDB
DTOs	Returning Mongo entities directly
Service layer	Large business logic blocks inside controllers
Environment variables	Hard-coded passwords/secrets
Postman + automated tests	Only manual browser testing
Git branches/commits	Keeping the whole project in one unstructured folder

17. Optional Admin Portal Clarification

The supplied deck lists 'MongoDB + JSP/Servlet' for a separate admin portal. [filecite2turn0file0L219-L223](#)

Recommended academic interpretation: React remains the primary candidate/employer frontend. If the faculty specifically requires JSP/Servlet practice, keep it isolated as an admin portal rather than mixing JSP into the

React application. The admin portal can consume secured REST APIs or be implemented as a separate module with carefully controlled access.

If JSP/Servlet is not a course requirement, it can be omitted to reduce complexity and keep the architecture consistently React + Spring Boot + MongoDB.

18. Candidate Workflow

- Register → verify email/account → complete Passport → add skills/projects/evidence → set availability → set visibility → become discoverable → receive invitation → view score breakdown → accept/decline/negotiate → message employer.

Post-match extension: after acceptance, the candidate enters the Project Workspace, receives milestone assignments, submits versioned deliverables, tracks project health, and receives milestone-linked payments after approval.

19. Employer Workflow

- Register → complete employer profile → create opportunity → define required/preferred skills → discover candidates → filter/re-rank → inspect match explanation → send invitation → candidate response → message → close opportunity.

Post-match extension: the employer defines milestones and budget weights, funds escrow, reviews deliverable versions, releases approved milestone payments, can publish paid Challenges, and may raise/resolve disputes through the governed workflow.

20. Admin Workflow

- Admin login → dashboard → review users/opportunities → inspect verification/moderation issues → take action → record audit event → monitor metrics → review matching/bias test results.

Post-match extension: admins monitor project health, escrow/dispute states, challenge moderation, evidence trails and audit events.

21. Testing Strategy

Test level	Examples
Unit	Matching score, DTO validation, permission rules, service methods
Integration	Controller + service + repository + test MongoDB
API	Postman auth/passport/opportunity/discovery/match/message flows
UI	Login, Passport completion, discovery, invitation, messaging
Security	Invalid JWT, wrong role, hidden passport, malformed payload
End-to-end	Candidate registration → employer discovery →

22. Deployment / Running Blueprint

- MongoDB: local MongoDB or MongoDB Atlas.
- Backend: configure application.properties or environment variables; run Maven/Spring Boot.
- Frontend: install npm dependencies and start React dev server.
- Browser: open frontend URL; React calls backend REST API.
- Typical development ports: React 5173 and Spring Boot 8080; configure CORS.
- Optional Windows start.bat can launch both applications after validating Java, Node and MongoDB availability.

Port numbers are implementation recommendations, not requirements from the supplied deck.

23. start.bat Concept

A simple Windows launcher can open two terminal windows: one for the Spring Boot backend and one for the React frontend. It should not contain secrets. A production deployment should use a proper process manager/container rather than relying on a batch file.

24. MVP Scope

The source deck defines the MVP as registration and Passport builder, employer portal and opportunity posting, rule-based matching, secure messaging and admin oversight. [filecite?url=turn0file0L299-L308](#)

- Registration + authentication
- Talent Passport
- Opportunity CRUD
- Discovery search/filter
- Rule-based matching
- Explainable match report
- Invitation + messaging
- Admin oversight
- Post-Match Governance: Project Workspace + Milestone Tracker
- Escrow Vault + versioned Deliverables
- Project Health Engine: Green / Yellow / Red
- Employer Challenges and Challenge Submissions
- Dispute Resolution and audit trail

25. Future Roadmap

- 6–12 months: GitHub OAuth verification, AI-assisted resume/skill extraction, predictive match scoring, mobile alerts.
- 12+ months: institutional SSO, university credential feeds, placement analytics, full mobile app.

These phases match the roadmap presented in the source deck. [filecite2turn0file0L301-L308](#)

26. Risks & Mitigation

Risk	Mitigation
Cold start	Seed with campus placement partners
Verification gaps	Use multiple evidence types; cap category weights
Privacy concerns	Default-private passports and explicit consent
Matching bias	Explainable rules + pre-pilot distribution checks
Scope creep	Freeze MVP before adding AI/mobile/institutional integrations
Integration failure	Keep GitHub and external verification adapters modular
Cold-start revenue	Use paid Employer Challenges to create immediate work opportunities with a small candidate pool.
Project delivery risk	Use milestones, versioned deliverables, escrow and health status rather than relying on informal messaging.
Payment/dispute risk	Maintain milestone-linked escrow states and admin-mediated evidence-based dispute resolution.

27. Success Metrics

- ≥60% Talent Passport completion target among registered candidates.
- ≥30% employer response rate to opportunity invitations.
- Track median time from employer search to first candidate contact.
- Track candidate-reported clarity of match feedback.
- Pilot with one campus placement cell + 10–15 partner employers.

The source states these are MVP validation targets, not current results. [filecite2turn0file0L277-L291](#)

28. Build Order / Development Phases

Phase	Deliverable
1	Repository, environment, React + Spring Boot skeleton
2	Mongo models, repositories, authentication
3	Talent Passport and evidence
4	Employer opportunity CRUD + discovery
5	Matching engine + explainability
6	Invitations + messaging
7	Admin oversight + audit
12	Testing, deployment/demo script and documentation
8	Post-Match Governance: Project Workspace + Milestone Tracker
9	Escrow Vault + Versioned Deliverables + Health Engine
10	Employer Challenges + Challenge Submissions
11	Dispute Resolution

29. Deliverables Checklist

- Source code repository
- React frontend
- Spring Boot backend
- MongoDB database setup / seed data
- Postman collection
- Test cases and results
- start.bat demo launcher
- Architecture diagram
- Database schema diagram
- API documentation
- Faculty PPT
- Full project documentation
- Demo script

30. Final Architecture at a Glance

Browser → React UI → REST/JSON → Spring Boot Controller → Service Layer → Repository → MongoDB

Security cross-cutting layer: Spring Security + JWT + BCrypt + validation + CORS.

Verification adapters: GitHub / certifications / assessments can plug into the Passport service without changing the core UI contract.

Matching engine: Opportunity criteria + Passport evidence → weighted score + per-skill explanation → invitation.

Post-match governance flow: Accepted Match → Project Contract → Milestones → Escrow → Deliverable Versions → Approval → Payment Release → Health/Dispute → Completion & Review.

31. Final Recommendation

Build the discovery loop first, then demonstrate the governance layer. The defensible academic MVP should show secure registration, complete Passport, employer opportunity, candidate discovery, transparent match score, invitation, response, messaging, and—where implemented—Project Workspace, milestones, escrow, deliverables, health status and Employer Challenges. Advanced AI/mobile/institutional integrations remain roadmap items.

This approach keeps the project defensible for faculty review while preserving the original TALENTX vision.

Appendix A — Suggested Naming Convention

- Java classes: PascalCase
- Java methods/variables: camelCase
- React components: PascalCase.jsx
- React utilities/services: camelCase.js
- Mongo collection names: snake_case plural

- REST paths: lowercase plural nouns
- DTO suffixes: Request / Response
- Environment variables: UPPER_SNAKE_CASE

Appendix B — Demo Scenario

- Create candidate account: Anika R.
- Complete Passport with Java, React, System Design and project evidence.
- Create employer opportunity requiring Java + React.
- Open Discovery and filter by skills.
- Show a high match with score breakdown.
- Send invitation.
- Switch to candidate account.
- Accept / decline / negotiate.
- Open messaging.
- Show admin audit/oversight view.

The supplied deck uses an illustrative candidate example 'Anika R. — Backend Engineer' with a 94% match to demonstrate the intended wow moment. [filecite:turn0file0L172-L198](#)

Updated demo lifecycle: after the invitation is accepted, the system creates a governed Project Workspace with milestone weights, escrow state, deliverable versions, health status and optional Challenge activity.

Illustrative Challenge flow: employer posts a \$100–\$500 micro-task, TALENTX surfaces it to suitable candidates, a candidate submits within the defined window, the employer selects a winner, and the platform records the payout/service-fee event.

- Create Project Contract after acceptance.
- Add three milestones, for example 20% / 30% / 50%, with deadlines.
- Lock milestone-linked funds in escrow.
- Upload deliverable v1, request revision, then upload v2 and approve.
- Show Green / Yellow / Red project health.
- Open a paid Employer Challenge and submit a solution.
- Complete the project, release remaining escrow, and record reviews/audit events.

7A. EXPANDED PROJECT FOLDER BLUEPRINT — MASTER LAYOUT

Purpose. This expanded layout turns the original high-level architecture into a practical file-by-file blueprint. It keeps React as the main Candidate/Employer UI, Spring Boot as the REST/business layer, MongoDB as the database, and the separate JSP/Servlet Admin portal only where the academic scope requires it. The supplied deck explicitly describes React → Spring Boot → MongoDB and a separate admin portal. [filecite](#) [turn2file0](#) [L11-L25](#)

7A.1 Master Repository Structure

```
TALENTX/
|
|— README.md
|— .gitignore
|— .env.example
|— start.bat
|— docker-compose.yml          # optional, for local MongoDB/services
|
|— frontend/                  # React Candidate + Employer application
|   |— package.json
|   |— vite.config.js
|   |— index.html
|   |— public/
|   |   |— favicon.ico
|   |   |— images/
|   |— src/
|   |   |— main.jsx
|   |   |— App.jsx
|   |   |— routes/
|   |   |— pages/
|   |   |— components/
|   |   |— layouts/
|   |   |— services/
|   |   |— context/
|   |   |— hooks/
|   |   |— utils/
```

```

├── validations/
├── constants/
├── assets/
├── styles/
├── backend/                                # Spring Boot REST API
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/com/taletx/
│       │   └── resources/
│       └── test/
├── admin/                                  # OPTIONAL JSP/Servlet admin portal
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/taletx/admin/
│       └── webapp/
├── database/
│   ├── seed/
│   ├── indexes/
│   └── sample-data/
├── postman/
│   ├── TALENTX-API.postman_collection.json
│   └── TALENTX-local.postman_environment.json
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   ├── testing/
│   └── screenshots/
├── tests/
│   └── end-to-end/

```

7A.2 Root-Level Files — What Each One Does

README.md — project overview, prerequisites, installation, run commands, architecture summary, demo accounts and troubleshooting.

.gitignore — prevents node_modules, target/, .env, IDE metadata and generated files from entering Git.

.env.example — documents required environment variables without storing real secrets.

start.bat — optional Windows demo launcher that starts frontend and backend. It must never contain passwords, JWT secrets or API tokens.

docker-compose.yml — optional convenience file if the team chooses containerized MongoDB/local services.

Do not create random Java/React files directly in the repository root. Keep implementation inside frontend/, backend/, and optional admin/.

7A.3 Frontend — Full React Layout

```
frontend/
├── package.json
├── vite.config.js
├── index.html
├── public/
│   ├── images/
│   └── icons/
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── routes/
    │   ├── AppRoutes.jsx
    │   ├── ProtectedRoute.jsx
    │   └── RoleRoute.jsx
    ├── layouts/
    │   ├── AuthLayout.jsx
    │   ├── CandidateLayout.jsx
    │   ├── EmployerLayout.jsx
    │   └── ErrorLayout.jsx
    ├── pages/
    │   ├── auth/
    │   │   ├── Login.jsx
    │   │   ├── Register.jsx
    │   │   └── ForgotPassword.jsx
    │   ├── candidate/
    │   │   ├── CandidateDashboard.jsx
    │   │   ├── Passport.jsx
    │   │   ├── Skills.jsx
    │   │   ├── Projects.jsx
    │   │   ├── Certifications.jsx
    │   │   ├── Assessments.jsx
    │   │   ├── Internships.jsx
    │   │   ├── Invitations.jsx
    │   │   ├── MatchDetails.jsx
    │   │   └── CandidateSettings.jsx
    │   ├── employer/
    │   │   ├── EmployerDashboard.jsx
    │   │   ├── CreateOpportunity.jsx
    │   │   ├── ManageOpportunities.jsx
    │   │   ├── Discovery.jsx
    │   │   ├── CandidateProfile.jsx
    │   │   ├── MatchDetails.jsx
    │   │   └── EmployerMessages.jsx
    │   └── common/
    │       ├── Home.jsx
    │       ├── Unauthorized.jsx
    │       └── NotFound.jsx
    ├── components/
    │   ├── common/
    │   │   ├── Navbar.jsx
    │   │   ├── Sidebar.jsx
    │   │   ├── Button.jsx
    │   │   ├── Modal.jsx
    │   │   └── Loader.jsx
```

```

├── ErrorMessage.jsx
├── passport/
│   ├── PassportCard.jsx
│   ├── SkillBadge.jsx
│   ├── ProjectCard.jsx
│   └── VerificationBadge.jsx
├── matching/
│   ├── MatchScore.jsx
│   ├── SkillBreakdown.jsx
│   └── MatchExplanation.jsx
├── opportunity/
│   ├── OpportunityCard.jsx
│   ├── OpportunityForm.jsx
│   └── OpportunityFilters.jsx
├── messaging/
│   ├── ConversationList.jsx
│   ├── MessagePanel.jsx
│   └── MessageInput.jsx
├── services/
│   ├── api.js
│   ├── authService.js
│   ├── passportService.js
│   ├── opportunityService.js
│   ├── discoveryService.js
│   ├── matchingService.js
│   ├── messageService.js
│   └── adminService.js
├── context/
│   ├── AuthContext.jsx
│   └── NotificationContext.jsx
├── hooks/
│   ├── useAuth.js
│   ├── useFetch.js
│   └── useDebounce.js
├── validations/
│   ├── authValidation.js
│   ├── passportValidation.js
│   └── opportunityValidation.js
├── constants/
│   ├── roles.js
│   ├── routes.js
│   └── matchWeights.js
├── utils/
│   ├── tokenStorage.js
│   ├── formatDate.js
│   └── formatScore.js
├── assets/
└── styles/
    ├── globals.css
    └── variables.css

```

Frontend rule

Pages decide what screen to show. Components render reusable UI. Services call the backend. Context stores application-wide state. Validation handles client-side convenience checks, but the backend remains the final authority. Matching calculations must not be trusted when performed only in React.

7A.4 Backend — Full Spring Boot Layout

```
backend/ ├── pom.xml └── src/ ├── main/ │ │ ├── java/com/talentx/ │ │ │ ├── TalentXApplication.java │ │ │ ├── controller/ │ │ │ │ ├── AuthController.java │ │ │ │ ├── PassportController.java │ │ │ │ ├── OpportunityController.java │ │ │ │ ├── DiscoveryController.java │ │ │ │ ├── MatchingController.java │ │ │ │ ├── InvitationController.java │ │ │ │ ├── MessageController.java │ │ │ │ ├── ProjectController.java │ │ │ │ ├── MilestoneController.java │ │ │ │ ├── EscrowController.java │ │ │ │ ├── DeliverableController.java │ │ │ │ ├── ChallengeController.java │ │ │ │ ├── ChallengeSubmissionController.java │ │ │ │ ├── DisputeController.java │ │ │ │ ├── AdminController.java │ │ │ │ └── service/ │ │ │ │ ├── AuthService.java │ │ │ │ ├── PassportService.java │ │ │ │ ├── OpportunityService.java │ │ │ │ ├── DiscoveryService.java │ │ │ │ ├── MatchingService.java │ │ │ │ ├── InvitationService.java │ │ │ │ ├── MessageService.java │ │ │ │ ├── ProjectService.java │ │ │ │ ├── MilestoneService.java │ │ │ │ ├── EscrowService.java │ │ │ │ ├── DeliverableService.java │ │ │ │ ├── ChallengeService.java │ │ │ │ ├── ChallengeSubmissionService.java │ │ │ │ ├── HealthEngineService.java │ │ │ │ ├── DisputeService.java │ │ │ │ ├── NotificationService.java │ │ │ │ ├── AdminService.java │ │ │ │ └── repository/ │ │ │ │ ├── TalentPassportRepository.java │ │ │ │ ├── OpportunityRepository.java │ │ │ │ ├── MatchRepository.java │ │ │ │ ├── InvitationRepository.java │ │ │ │ ├── ConversationRepository.java │ │ │ │ ├── ProjectRepository.java │ │ │ │ ├── MilestoneRepository.java │ │ │ │ ├── MessageRepository.java │ │ │ │ ├── EscrowTransactionRepository.java │ │ │ │ ├── DeliverableRepository.java │ │ │ │ ├── ChallengeRepository.java │ │ │ │ ├── ChallengeSubmissionRepository.java │ │ │ │ ├── DisputeRepository.java │ │ │ │ └── AuditLogRepository.java │ │ │ │ ├── model/ │ │ │ │ ├── User.java │ │ │ │ ├── TalentPassport.java │ │ │ │ ├── Opportunity.java │ │ │ │ ├── Match.java │ │ │ │ ├── Invitation.java │ │ │ │ ├── Conversation.java │ │ │ │ ├── Message.java │ │ │ │ ├── Project.java │ │ │ │ ├── Milestone.java │ │ │ │ ├── EscrowTransaction.java │ │ │ │ ├── Deliverable.java │ │ │ │ ├── Challenge.java │ │ │ │ ├── ChallengeSubmission.java │ │ │ │ ├── Dispute.java │ │ │ │ └── AuditLog.java │ │ │ │ ├── dto/ │ │ │ │ ├── auth/ │ │ │ │ ├── milestone/ │ │ │ │ ├── candidate/ │ │ │ │ ├── escrow/ │ │ │ │ ├── employer/ │ │ │ │ ├── deliverable/ │ │ │ │ ├── challenge/ │ │ │ │ ├── mapper/ │ │ │ │ ├── security/ │ │ │ │ ├── matching/ │ │ │ │ ├── verification/ │ │ │ │ ├── integration/github/ │ │ │ │ ├── validation/ │ │ │ │ ├── exception/ │ │ │ │ ├── config/ │ │ │ │ ├── resources/ │ │ │ │ ├── application.properties │ │ │ │ ├── application-dev.properties │ │ │ │ ├── test/ │ │ │ │ └── java/com/talentx/ │ │ │ │ └── service/ │ │ │ │ └── controller/ │ │ │ │ └── security/
```

7A.5 Data / MongoDB Blueprint

```
database/ ├── seed/ │ │ ├── users.json │ │ ├── passports.json │ │ └── opportunities.json ├── sample-data/ │ └── demo-data.json └── indexes/ └── create-indexes.js
```

Collection	Main fields	Used by	Do not store
users	email, passwordHash, role, status	Auth/Admin	plain password, raw JWT
talent_passports	userId, skills, projects, evidence, visibility	Candidate/Discovery	private fields when not permitted
opportunities	employerId, requiredSkills, preferredSkills, status	Employer/Matching	unvalidated external input
matches	candidateId, opportunityId, score, breakdown	Matching	only final score without explanation

messages	conversationId, senderId, receiverId, body, timestamp	Messaging	passwords/tokens
audit_logs	actor, action, target, timestamp	Admin/Security	unnecessary personal data
projects	matchId, employerId, freelancerId, totalBudget, status, healthScore	Project Workspace	unnecessary personal data
milestones	projectId, deadline, monetaryWeight, status, order	Project tracking	raw payment credentials
escrow_transactions	projectId, milestoneId, amount, status, releaseDate	Escrow	plain secrets
deliverables	milestoneId, version, fileUrl, revisionNotes, status	Delivery	unrelated private data
challenges	employerId, requiredSkills, prizeAmount, deadline, status	Challenge engine	unnecessary personal data
challenge_submissions	challengeId, candidateId, solutionUrl, submittedAt, isWinner	Challenge workflow	raw credentials
disputes	projectId, raisedBy, reason, evidenceUrls, resolution	Admin resolution	unnecessary personal data

7A.6 Optional JSP/Servlet Admin Portal

```

admin/
├── pom.xml
└── src/main/
    ├── java/com/taletx/admin/
    │   ├── servlet/
    │   │   ├── AdminLoginServlet.java
    │   │   ├── UserManagementServlet.java
    │   │   ├── VerificationServlet.java
    │   │   └── AuditServlet.java
    │   └── filter/
    │       └── AdminAuthFilter.java
    └── webapp/
        ├── WEB-INF/
        │   └── web.xml
        ├── login.jsp
        ├── dashboard.jsp
        ├── users.jsp
        ├── verification.jsp
        └── audit.jsp

```

This module is optional. The supplied deck describes a separate admin portal outside the main React application; therefore this folder should not be mixed into frontend/src. [filecite2turn2file02L21-L252](#)

7A.7 API and Documentation Layout

```
docs/
├── architecture/
│   ├── system-architecture.md
│   ├── component-diagram.png
│   └── data-flow.png
├── api/
│   ├── endpoints.md
│   └── error-codes.md
├── database/
│   ├── collections.md
│   └── erd-equivalent.md
├── testing/
│   ├── test-plan.md
│   └── test-results.md
└── screenshots/
    ├── candidate/
    ├── employer/
    └── admin/
```

7A.8 File Dependency Flow

```
React Page
↓
React Component
↓
Service file (HTTP request)
↓
Spring Boot Controller
↓
DTO validation
↓
Service layer
↓
Repository
↓
MongoDB
```

Security runs across the API:
JWT → Authentication Filter → Role Authorization → Controller

```
Matching:
Opportunity + Talent Passport
↓
MatchingService
↓
ScoreCalculator
↓
Match document
↓
Explainable MatchResponse
↓
React MatchDetails page
```